

Department of Computer Science and Engineering
22CS74 – Cloud Computing
Unit 1: Cloud Computing & Virtualization – Lecture Notes

Introduction to Cloud Computing

Cloud Computing at a Glance

- **Vision of Computing Utilities:** Computing is being transformed into a model where services are **commoditized and delivered like utilities** such as water, electricity, gas, and telephony. Users access services based on requirements, regardless of hosting location. Cloud computing is the latest paradigm aiming to make "computing utilities" a reality.
- **Definition:** Cloud computing is a technological advancement focusing on designing computing systems, developing applications, and leveraging existing services for software building. It is based on **dynamic provisioning** of services, compute capability, storage, networking, and IT infrastructure generally. Resources are made available over the Internet on a **pay-per-use basis** from vendors.
- **Accessibility and Flexibility:** Anyone with a credit card can subscribe to cloud services, deploy and configure servers in hours, grow and shrink infrastructure based on demand, and pay only for resource usage time. Users don't need to invest heavily or maintain complex IT infrastructure.
- **Historical Anticipation:** Leonard Kleinrock in 1969 anticipated the spread of "computer utilities". John McCarthy in 1961 also foresaw computing organized as a public utility.
- **Core Principle:** Cloud computing turns IT services into utilities. This is enabled by the maturity of technologies like Web 2.0 (Internet as a rich application platform), service orientation (familiar abstractions), and virtualization (customization, control, flexibility).
- **Key Advantage:** It allows integrating additional capacity or features into existing systems, which is more attractive than buying new infrastructure whose sizing is hard to estimate and needs are limited in time. This has made it a popular phenomenon.
- **Long-Term Vision:** The long-term vision is an **open environment where computing, storage, and other services are traded as computing utilities in a global digital market**. This "cloud marketplace" would enable automated discovery and integration of services into existing software, reducing barriers between consumers and providers.

- **Benefits from Consolidation:** Concentrating IT capabilities into large datacenters optimizes facilities, fully utilizes capabilities for multiple users, reduces energy waste, lowers carbon emissions (contributing to greener IT), and increases revenue.

- **Different Views:** Cloud computing is a buzzword associated with various concepts like virtualized infrastructure, utility computing, IT outsourcing, PaaS, SaaS, etc.. Despite varied perceptions, the common element is **dynamic provisioning of IT services with a utility-based cost model**.

Cloud computing is computing delivered as a utility (like electricity or water). It allows users to access computing resources on-demand with pay-as-you-go pricing. The vision originated from Leonard Kleinrock's ARPANET utility computing idea (1969).

1.1.2 Defining a Cloud

- **Internet-Centric:** The term "cloud" historically represented networks, and in cloud computing, it signifies an Internet-centric way of computing, as the Internet is the medium and platform for service delivery.

- **Armbrust et al. Definition:** Cloud computing refers to "both the applications delivered as services over the Internet and the hardware and system software in the datacenters that provide those services". This encompasses the entire stack, from hardware to high-level applications, introducing "Everything as a Service" (XaaS).

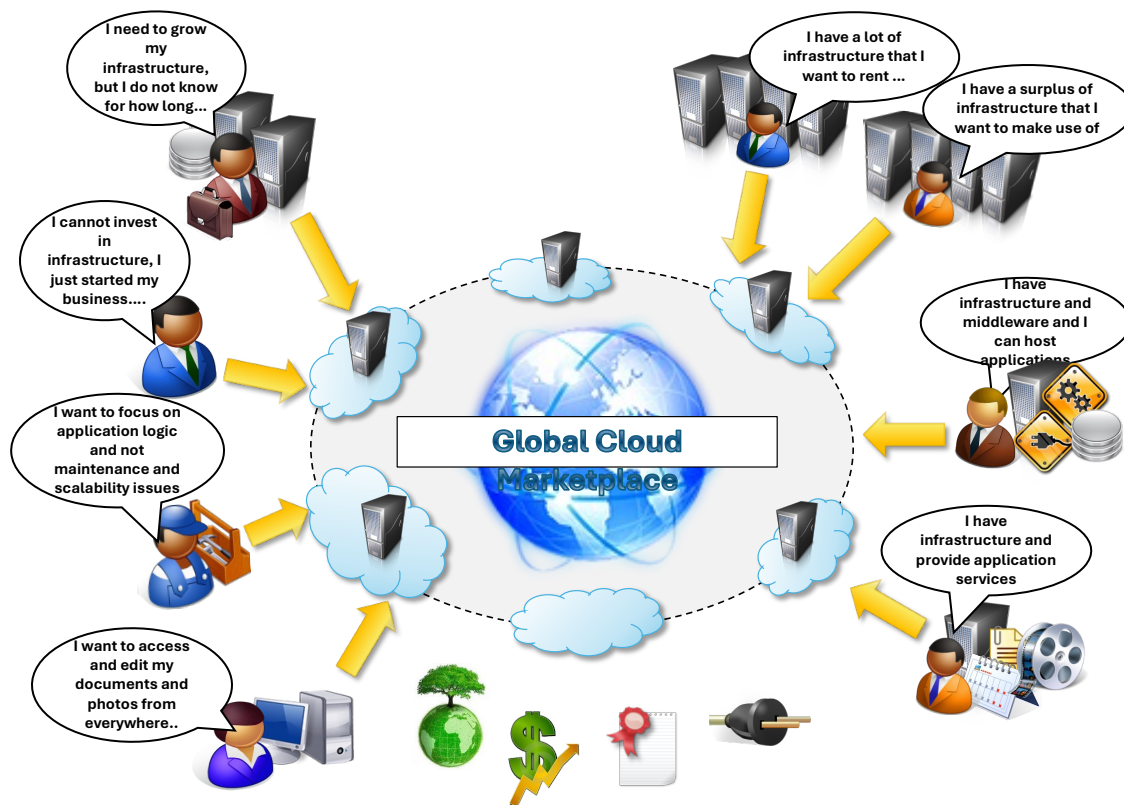
- **NIST Definition:** Cloud computing is "a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction".

- **Utility-Oriented Approach:** It focuses on delivering services with a **pay-per-use pricing model**. Reese defines three criteria for cloud services:

- Accessible via a **Web browser (nonproprietary) or Web services API**.
- **Zero capital expenditure** necessary to start.
- **Pay only for what you use** as you use it.

- **Buyya et al. Definition:** "A cloud is a type of parallel and distributed system consisting of a collection of interconnected and virtualized computers that are dynamically provisioned and presented as one or more unified computing resources based on service-level agreements established through negotiation between the service provider and consumers".

Diagram: Cloud Vision



The Vision of Cloud Computing

Cloud computing envisions an open marketplace where IT services are traded as utilities. It enables scalability, elasticity, and democratization of computing resources.

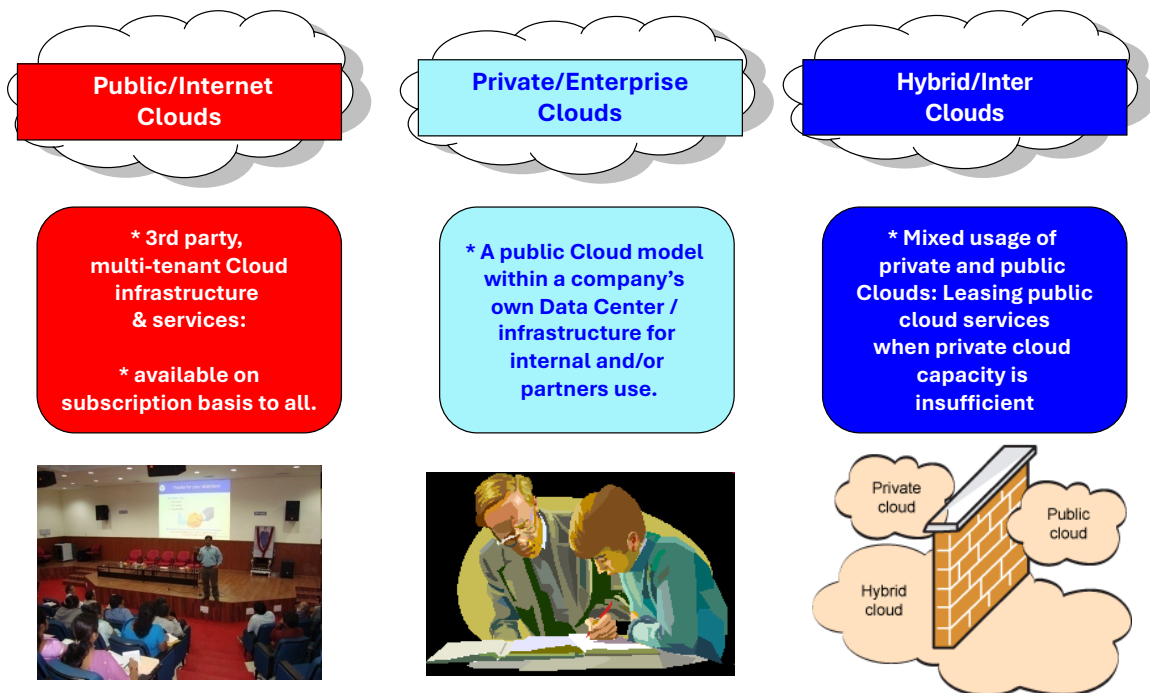
Defining a Cloud; A Closer Look

A Closer Look (Practical Examples)

Cloud computing accommodates various scenarios by providing on-demand IT services on a pay-as-you-go basis:

- **Large enterprises:** Can offload activities. The New York Times converted its digital library using Amazon EC2 and S3 for a short period, then relinquished resources with no additional costs.

- **Small enterprises and start-ups:** Can translate ideas into business results quickly without excessive up-front costs. Animoto scaled from 70 to 8,500 servers in one week using Amazon Web Services, owning no servers.
- **System developers:** Can focus on business logic rather than infrastructure management. Little Fluffy Toys developed a widget on Google AppEngine and was on the market in one week.
- **End users:** Can access documents and data anytime, anywhere, from any device. Apple iCloud stores documents in the cloud, allowing seamless access and editing across devices (smartphone, laptop, tablet) without physical connections.

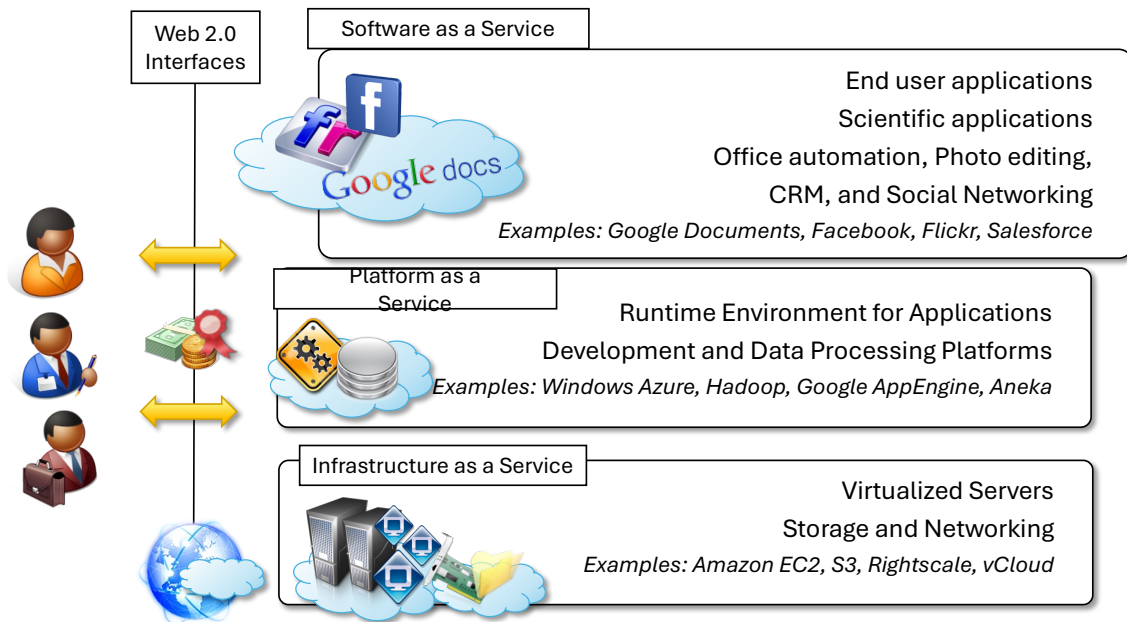


Deployment Models

- **Public Clouds:** Most common, where IT infrastructure (e.g., virtualized datacenters) is established by a third-party provider and made available to any consumer on a subscription basis. Users' data and applications are deployed on the vendor's premises.
- **Private/Enterprise Clouds:** Large organizations replicate the cloud IT service delivery model in-house. Driven by the need to keep confidential information within premises, preferred by governments and banks with high security and privacy concerns.
- **Hybrid Clouds:** Composed of public cloud resources and privately owned infrastructures. Used when private cloud resources cannot meet QoS requirements, offering a common way to explore cloud possibilities.

Cloud Computing Reference Model

The Cloud Computing Reference Model consists of 3 service layers: IaaS, PaaS, SaaS.



The Cloud Computing Reference Model

Classifies cloud computing service offerings into three major categories, forming a layered view of the computing stack:

- **Infrastructure-as-a-Service (IaaS):** Delivers infrastructure on demand (virtual hardware, storage, networking) at the base of the stack.

- **Virtual hardware:** Provides compute on demand as virtual machine instances, configured by users, typically priced per hour. Examples: Amazon EC2, Rightscale, vCloud.

- **Virtual storage:** Raw disk space or object store. Examples: Amazon S3.

- **Virtual networking:** Manages networking among virtual instances and connectivity.

- **Use Case:** Users building dynamically scalable computing systems requiring specific software stacks, scalable websites, or background processing.

- **Platform-as-a-Service (PaaS):** Delivers scalable and elastic runtime environments on demand for application execution.

- **Provider Responsibility:** Service provider manages scalability and fault tolerance.

- **User Responsibility:** Users focus on application logic, leveraging provider's APIs and libraries.
- **Abstraction:** Increases abstraction level but constrains user in a controlled environment. Examples: Windows Azure, Hadoop, Google AppEngine, Aneka.
- **Use Case:** Appropriate for developing new systems that require a scalable programming platform.
- **Software-as-a-Service (SaaS):** Provides applications and services on demand at the top of the stack.
 - **Functionality:** Common desktop application functionalities (office automation, CRM, photo editing) replicated on provider's infrastructure, accessible via browser.
 - **Sharing:** Applications shared across multiple isolated users.
 - **Examples:** Google Documents, Facebook, Flickr, Salesforce.
 - **Use Case:** Targets end users benefiting from elastic scalability without software development, installation, configuration, or maintenance. Suitable when existing SaaS fits needs with minimal customization.

Characteristics and Benefits

Key benefits: no upfront commitments, elasticity, efficient resource use, energy efficiency, reduced CapEx, operational efficiency, agility.

- **Characteristics:** No up-front commitments, on-demand access, nice pricing, simplified application acceleration and scalability, efficient resource allocation, energy efficiency, seamless creation and use of third-party services.

- **Economic Benefits:**

- **Increased Economical Return:** Due to reduced maintenance and operational IT costs. IT assets become utility costs, paid only for use, not up front.

- **Reduced Capital Costs:** Eliminates large up-front investments in IT infrastructure and software, which are depreciable. This allows small organizations and start-ups to grow comfortably without massive initial outlays.

- **Reduced Maintenance Costs:** Responsibility shifts to the cloud service provider, who benefits from economies of scale.

- **Operational Benefits:**

- **Increased Agility:** Dynamic and flexible composition of software systems.

- **Reduced Capacity Planning:** Organizations can react rapidly to unplanned surges in demand (e.g., adding/dismissing servers for workload spikes).

- **Ease of Scalability:** Leverage potentially huge cloud capacity to extend IT capability across the entire computing stack (IaaS, PaaS, SaaS offerings).

- **End-User Benefits:** Data and processing capabilities are always available from anywhere, anytime, through multiple devices, via Web-based interfaces. Eliminates the need for considerable software investments for tasks like office automation or photo editing.

- **Efficiency and Environmental Benefits:** Multitenancy allows better utilization of shared infrastructure. Concentration of IT infrastructure in large datacenters enables considerable optimization in resource allocation and **energy efficiency**, contributing to **greener IT**.

- **New Opportunities:** Service orientation and on-demand access allow creating new service offerings by aggregating existing ones and focusing on added value with limited costs.

Challenges Ahead

Challenges include: security, privacy, compliance, legal issues due to geo-distribution, interoperability, standardization, and technical provisioning.

- **Dynamic Provisioning Challenges:** Determining how many resources to provision and for how long to maximize benefit.

- **Technical Challenges for Providers:** Managing large computing infrastructures and virtualization technologies on top of them.

- **Integration Issues:** Integrating real and virtual infrastructure, especially concerning security and legislation.

- **Security:**

- **Confidentiality, Secrecy, Data Protection:** Organizations don't own the infrastructure, raising concerns for confidential data. Assurance on data confidentiality and compliance to security standards are crucial.

- **Weak Point:** Data needs to be decrypted in memory for processing, and virtualization allows malicious providers to capture memory pages transparently.

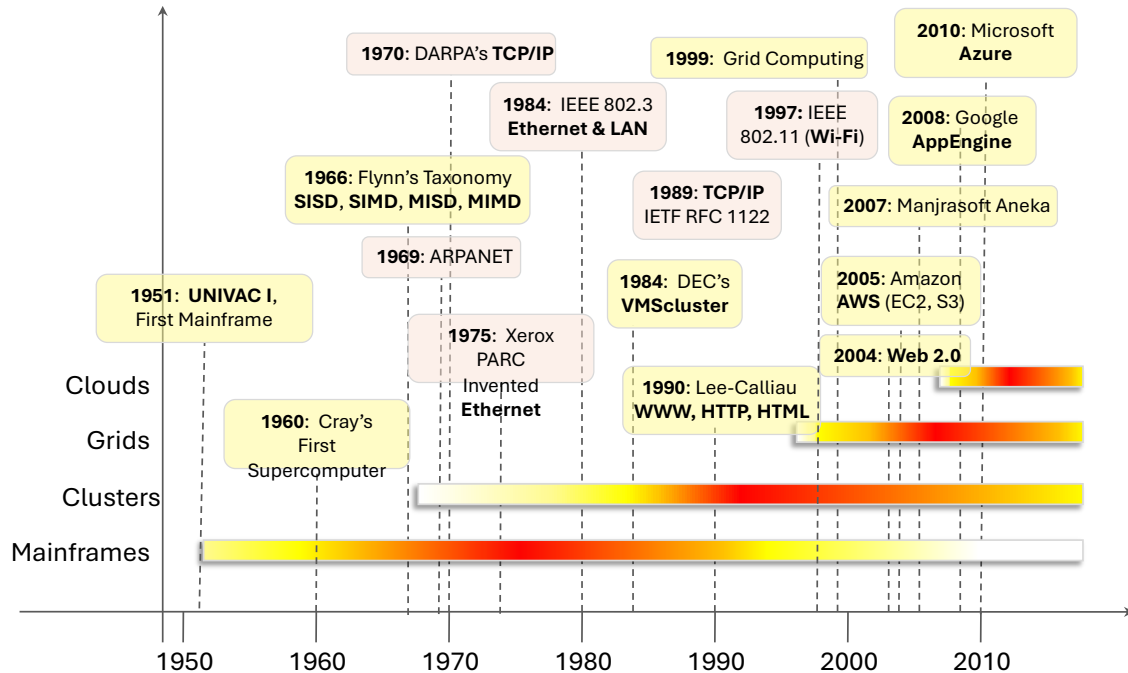
- **Legal Issues:**

- **Ubiquitous Nature:** Cloud infrastructure spreads across diverse geographical locations.

- **Jurisdictional Disputes:** Different privacy legislations in various countries can create disputes regarding third-party access (e.g., government agencies) to data. Example: US

organizations storing data in Europe, where European laws protect privacy more strictly than US laws on national security grounds.

Historical Developments



Cloud computing is a culmination of several core distributed computing technologies.

- **Distributed Systems:** Clouds are essentially large distributed computing facilities. A distributed system is a "collection of independent computers that appears to its users as a single coherent system". They share resources and utilize them better. Clouds exhibit properties like scalability, concurrency, and continuous availability.

- **Mainframes (1950s):** First large computational facilities using multiple processing units, highly reliable, "always on". Used for bulk data processing; evolved versions still used for transaction processing. Offered computing power as a service by providers like IBM.

- **Cluster Computing (1980s):** Low-cost alternative to mainframes and supercomputers, connecting cheap commodity machines with high-bandwidth networks and managed as a single system. Made high-performance computing widely accessible. Contributed tools like Condor, PVM, MPI.

- **Grid Computing (early 1990s):** Evolution of clusters, proposing access to computational power, storage, and services like utilities (power, gas, water). Aggregated geographically dispersed clusters from different organizations. Dynamic aggregation of heterogeneous nodes, nationwide or worldwide scale. Improvements in networking and Internet diffusion enabled it. Brought concept of "market orientation" for trading computing resources.

- **Relationship to Cloud:** Cloud computing embodies aspects of all three: deployed in large datacenters by single organizations (like mainframes), virtually infinite capacity, fault-tolerant, always on. Uses commodity machines (like clusters). Services consumed on a pay-per-use basis, fully implementing the utility vision introduced by grids.

- **Virtualization:** Core technology allowing abstraction of hardware, runtime environments, storage, and networking. Overcame past limitations on efficiency, now fundamental for cloud computing. Confers customization and control for users, sustainability for providers.

- **Web 2.0 (around 2004):** The primary interface for cloud services. Transformed the Web into a rich platform for application development, facilitating interactive information sharing, collaboration, user-centered design, and application composition. Brings interactivity, flexibility, enhanced user experience (e.g., AJAX, Web Services, XML). Enabled dynamic applications with continuous updates and features without client-side deployments. Promotes loose coupling and composition of services. Examples: Google Documents, Flickr, Facebook. Made people accustomed to using the Internet for everyday life and accepting IT infrastructure delivery via Web interface.

- **Service-Oriented Computing (SOC):** Core reference model for cloud systems, using services as main building blocks. Supports rapid, low-cost, flexible, interoperable, and evolvable applications.

- **Service Definition:** Self-describing, platform-agnostic component performing a function, exposing functionalities via network-accessible protocols. Loosely coupled, reusable, programming language independent, location transparent.

- **Service-Oriented Architecture (SOA):** Logical organization of software systems providing services through published and discoverable interfaces.

- **Key Concepts in SOC:**

- **Quality of Service (QoS):** Functional and nonfunctional attributes (performance, security, reliability, scalability, availability) to evaluate service behavior, established via a Service-Level Agreement (SLA).

- **Software-as-a-Service (SaaS):** New delivery model inherited from Application Service Providers (ASPs), where software is delivered over WAN on subscription/rental basis. ASP maintains infrastructure, client is freed from costs/upgrades. Enabled by **multitenancy** for economies of scale. SOC allows exposing and pricing loosely coupled components singularly.

- **Web Services (WS):** Popular expression of service orientation, making Web consumable by applications, not just humans. Expose functionalities via HTTP, interface inferred by WSDL (XML for service characteristics), interaction via SOAP (XML for method invocation/results). Platform independent and accessible to WWW.

- **Utility-Oriented Computing:** Vision of computing where resources (storage, compute power, applications, infrastructure) are packaged and offered on a pay-per-use basis. John McCarthy's 1961 vision. Traces in mainframe era (IBM offering mainframe power). Extended to academia with cluster computing. Internet and Web enabled worldwide scale. E-commerce technologies (online payments) provided infrastructure support. SOC broadened what could be accessed as a utility (services, application components).

Timeline of Computing Evolution

- → 1950s – Mainframes
- → 1980s – Clusters
- → 1990s – Grids
- → 2000s – Clouds

Building Cloud Computing Environments

- **Application Development:** Cloud computing benefits applications that need **dynamic scaling on demand**.

- **Web Applications:** Performance influenced by varying user demands, often complex multi-tier applications susceptible to inappropriate infrastructure sizing or workload variability.

- **Resource-Intensive Applications:** Data-intensive or compute-intensive, requiring considerable resources for short periods (e.g., scientific applications for large-scale experiments). Cloud computing offers a solution instead of buying infrastructure.

- **How Cloud Enables This:** Provides methods for renting compute/storage/networking; offers scalable runtime environments; provides application services mimicking desktop apps but hosted by provider. Leverages service orientation, accessible via simple Web interfaces (often REST Web services).

- **Infrastructure and System Development:** Requires knowledge across distributed computing, virtualization, service orientation, and Web 2.0.

- **Distributed Systems:** Cloud systems are distributed systems, with the major challenge being the **extreme dynamism** (new nodes/services provisioned on demand). IaaS offers resource addition/removal, PaaS embeds control algorithms for provisioning. Integration with existing systems is a concern.

- **Web 2.0 and Service Orientation:** Web 2.0 technologies are the interface for cloud services. Web services are primary access points programmatically. **Cloud computing is summarized as XaaS (Everything-as-a-Service)**, highlighting service orientation's central role.

- **Virtualization:** Core feature of provider infrastructure. Developers need to be aware of limitations of chosen virtualization technology and implications on system component volatility.

- **Design Considerations:** Dynamism, scale, and volatility of components should guide the design of cloud systems. Cloud computing provides mechanisms to address demand surges by replicating components under stress.

1.3.3 Computing Platforms and Technologies (Examples)

- **Amazon Web Services (AWS):** Offers comprehensive cloud IaaS services (virtual compute, storage, networking).

- **Elastic Compute Cloud (EC2):** Customizable virtual hardware instances for base infrastructure, various configurations (GPU, cluster instances), deployed via Web portal or Web services API. Allows saving running instances as images (templates).

- **Simple Storage Service (S3):** Delivers persistent storage on demand, organized into buckets containing objects (files, disk images) accessible globally.

- **Additional Services:** Networking, caching, DNS, relational/non-relational databases.

- **Google AppEngine:** Scalable runtime environment primarily for Web applications, leveraging Google's large infrastructure for dynamic scaling.

- **Features:** Secure execution environment, services for scalable/high-performance Web app development (in-memory caching, scalable data store, job queues, messaging, cron tasks).

- **Development:** SDK for local development/testing, easy migration to AppEngine, cost quotas. Supports Python, Java, Go.

- **Microsoft Azure:** Cloud operating system and platform for developing cloud applications.

- **Roles:** Applications organized around roles (distribution units embodying logic): Web role (hosts Web app), Worker role (generic container for workload processing), Virtual Machine role (fully customizable virtual environment including OS).

- **Additional Services:** Storage (relational, blobs), networking, caching, content delivery.

- **Apache Hadoop:** Open-source framework for processing large data sets on commodity hardware.

- **MapReduce Implementation:** Implementation of Google's MapReduce programming model (map for transforming/synthesizing input, reduce for aggregating output).

- **Usage:** Developers provide input data and specify map/reduce functions. Yahoo! is a major sponsor, uses Hadoop for its cloud infrastructure and business processes, manages the world's largest Hadoop cluster.

- **Force.com and Salesforce.com:** Force.com is a cloud computing platform for social enterprise applications.

- **Salesforce.com:** A SaaS solution for customer relationship management built on Force.com.

- **Development:** Allows creating applications by composing ready-to-use blocks and custom components, or integrating from AppExchange. Provides full support from data layout design to business rules, workflows, and UI definition. Fully cloud-hosted, functionalities accessible via Web services.

- **Manjrasoft Aneka:** Cloud application platform for rapid creation and seamless/elastic deployment of scalable applications on various cloud types.

- **Programming Models:** Supports tasks, distributed threads, and map-reduce abstractions.

- **Runtime Environment:** Distributed service-oriented runtime deployable on heterogeneous hardware (clusters, networked desktops, cloud resources). Dynamically integrates additional resources.

- **Services:** Manages scheduling, execution, accounting, billing, storage, QoS at runtime.

Virtualization

Virtualization abstracts hardware, storage, network, and runtime environments. It enables multiple virtual machines (VMs) on a single physical machine.

Introduction to Virtualization

- **Definition:** A broad umbrella of technologies and concepts providing an abstract environment (virtual hardware, operating system) to run applications. It creates a **secure, customizable, and isolated execution environment**, even for untrusted applications, without affecting others.

- **Mechanism:** A computer program (or software/hardware combo) emulates an execution environment separate from its host. Example: Running Windows OS on a virtual machine hosted by Linux OS.

- **Role in Cloud Computing:** Fundamental component of cloud computing, especially for infrastructure-based services, enabling **elastically scalable systems** and delivering customizable computing environments on demand.

- **Scope:** While often synonymous with **hardware virtualization** (crucial for IaaS), virtualization applies to operating system level, programming language level, application level, and also to **storage, memory, and networking**.

- **Reasons for Renewed Interest:**

- **Increased Performance & Computing Capacity:** Modern PCs and supercomputers have ample resources to host virtual machines with acceptable performance.

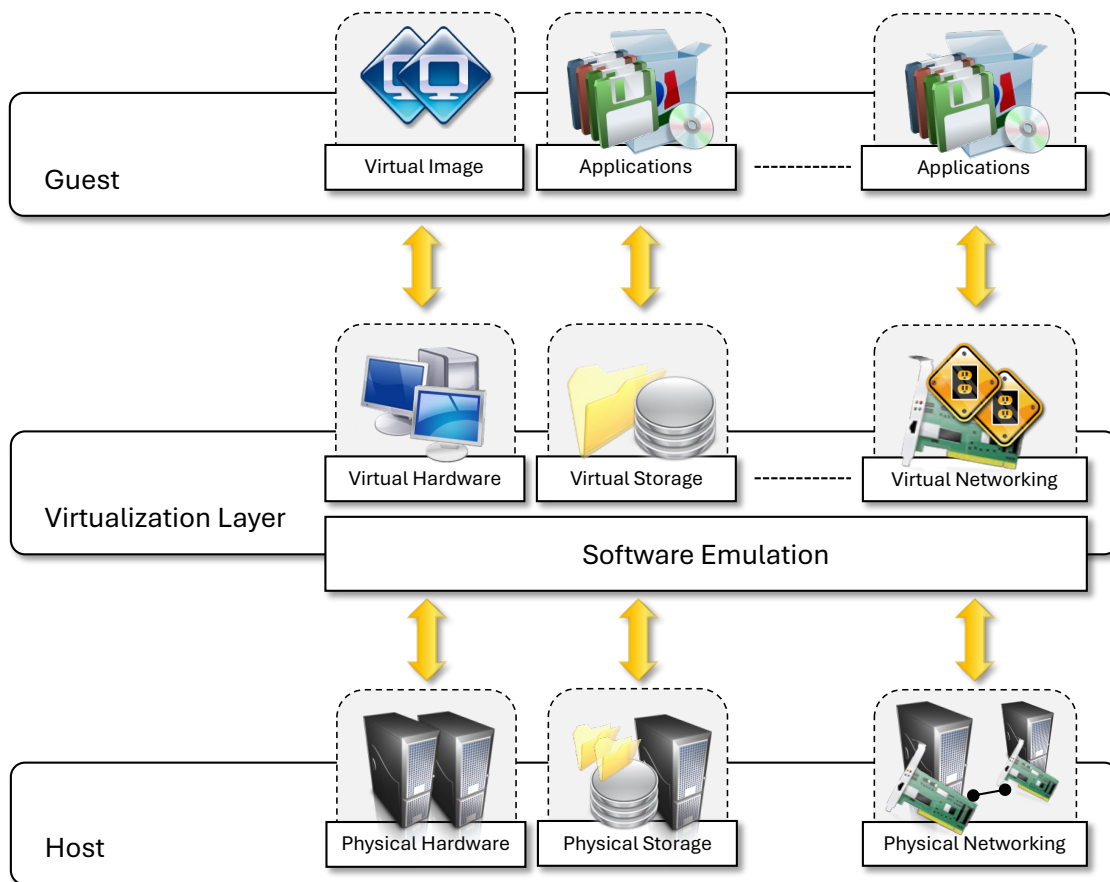
- **Underutilized Hardware/Software Resources:** Computers are often only partially utilized, especially enterprise IT infrastructure (e.g., desktop PCs unused overnight). Virtualization improves efficiency by utilizing these resources.

- **Lack of Space:** Data centers are growing rapidly, and companies seek ways to accommodate additional capacity without building new centers. This led to **server consolidation**, where virtualization is fundamental.

- **Greening Initiatives:** Data centers are major power consumers. Server consolidation through virtualization reduces the number of active servers, significantly cutting cooling and power consumption, thus reducing carbon footprint.

- **Rise of Administrative Costs:** Power and cooling costs exceed IT equipment costs. More servers mean higher administrative costs (monitoring, setup, updates, backups). Virtualization reduces server count, lowering labor costs.

- **Maturity of VM-based Programming Languages:** The popularity of Java (1995) and .NET Framework (2002), both based on virtual machine models, demonstrated that technology could support virtualized solutions without significant performance overhead, paving the way for more radical forms of virtualization.



Characteristics of Virtualized Environment

Characteristics: increased security, managed execution, portability, resource pooling, workload isolation.

• Components:

- **Guest:** The system component interacting with the virtualization layer.
- **Host:** The original environment where the guest is managed.
- **Virtualization Layer:** Recreates the environment for the guest to operate.
- *Example (Hardware Virtualization):* Guest is OS/applications, Host is physical hardware, Virtualization Layer is the Virtual Machine Manager (hypervisor).

• **Software-Based Emulation:** The virtual environment is created by a software program.

• Increased Security:

- **Controlled Execution:** Virtual machine manager controls and filters guest activity, preventing harmful operations. Resources from the host can be hidden or protected.

- **Isolation:** Sensitive host information is naturally hidden. Essential for untrusted code (e.g., Java applets running in a sandboxed JVM with limited resource access) [162, 163, 166n]. Hardware virtualization solutions (VMware Desktop, VirtualBox) provide completely separated file systems for guest OS from host.

- **Managed Execution:** Allows for a wider range of features:

- **Sharing:** Creates separate computing environments within the same host, fully exploiting powerful host capabilities that would otherwise be underutilized. Important in virtualized data centers for reducing active servers and power consumption.

- **Aggregation:** Groups separate hosts to be represented as a single virtual host to guests (e.g., cluster management software).

- **Emulation:** Controls and tunes the environment exposed to guests. Can emulate a completely different environment from the host, useful for testing across platforms or running legacy software on emulated hardware.

- **Isolation:** Provides guests with a completely separate environment, preventing interference between multiple guests on the same host and enabling filtering of guest activity to protect the host.

- **Performance Tuning:** Fine-tunes resource properties exposed via the virtual environment, enabling effective Quality of Service (QoS) infrastructure and fulfilling Service-Level Agreements (SLAs).

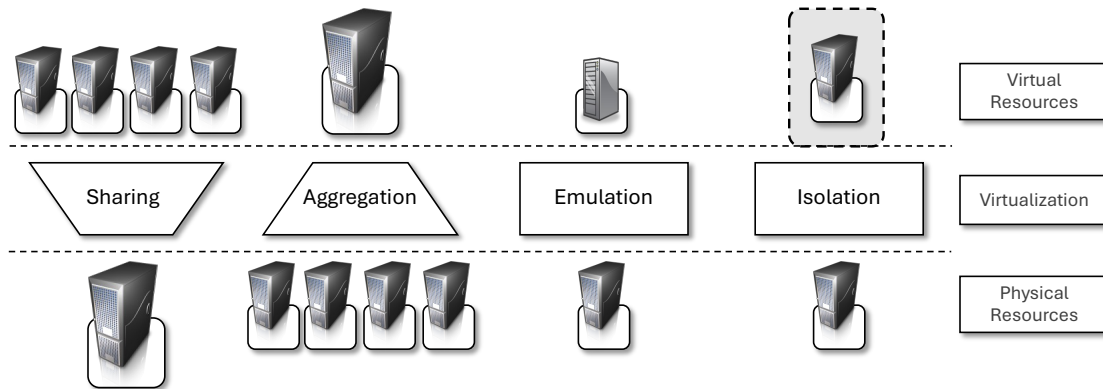
- **State Capturing and Migration:** Allows capturing the state of a guest program, persisting it, and resuming execution. **Virtual machine migration** enables moving a virtual image to another machine and resuming execution transparently. **Live migration** moves a running instance without interruption.

- **Portability:**

- **Hardware Virtualization:** Guest packaged into a virtual image that can be moved and executed on different virtual machines, similar to picture files. Proprietary formats often require specific VMM.

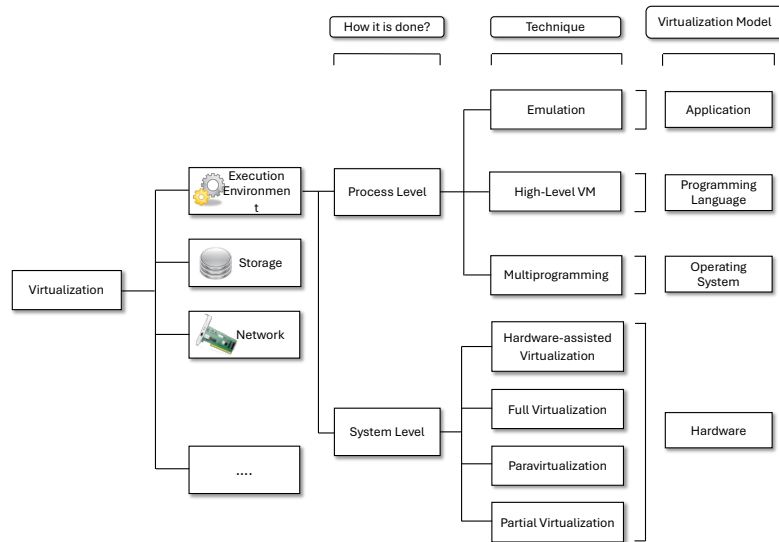
- **Programming-Level Virtualization:** Binary code (jars, assemblies) runs without recompilation on any implementation of the corresponding virtual machine. Simplifies development cycle and application deployment.

◦ **General:** Allows users to carry their own system ready to use, as long as the VMM is available.



Taxonomy of Virtualization Techniques

Virtualization techniques include execution virtualization (hardware, OS, programming language, application level), and others such as storage, network, desktop, and application server virtualization.



Virtualization enables elasticity, multitenancy, and dynamic provisioning – the foundation of cloud computing.

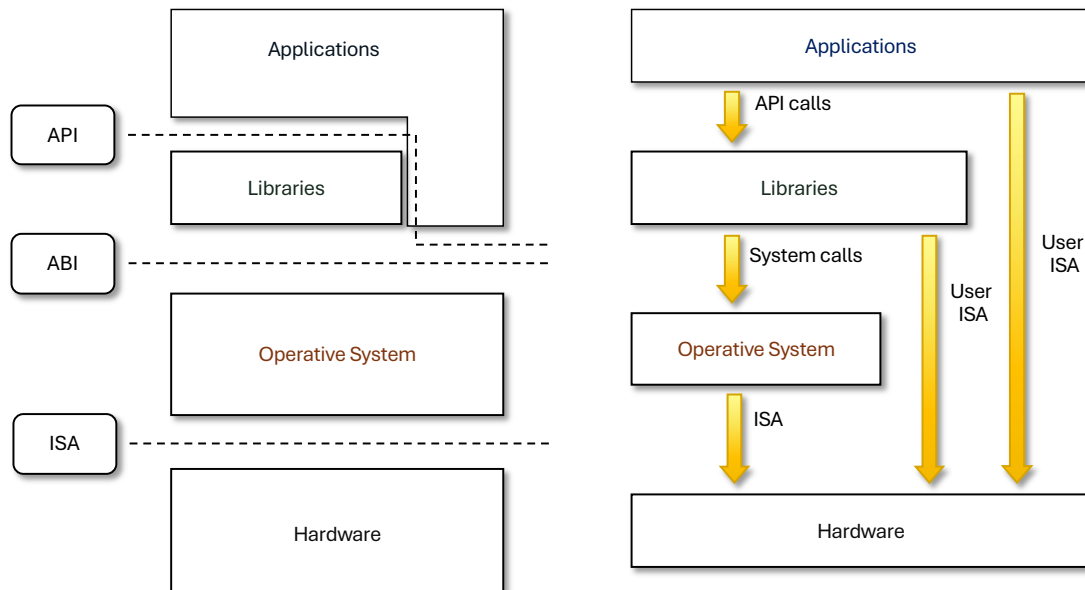
Virtualization techniques

Virtualization techniques are classified by the service/entity emulated and how it's done.

• **Main Categories:** Execution Environment, Storage, Network.

• **Execution Virtualization:** Emulates an execution environment separate from the host.

◦ **Machine Reference Model:** Modern computing systems have layers: Hardware (ISA), Operating System (ABI), Applications/Libraries (API). Virtualization techniques replace one of these layers and intercept calls.



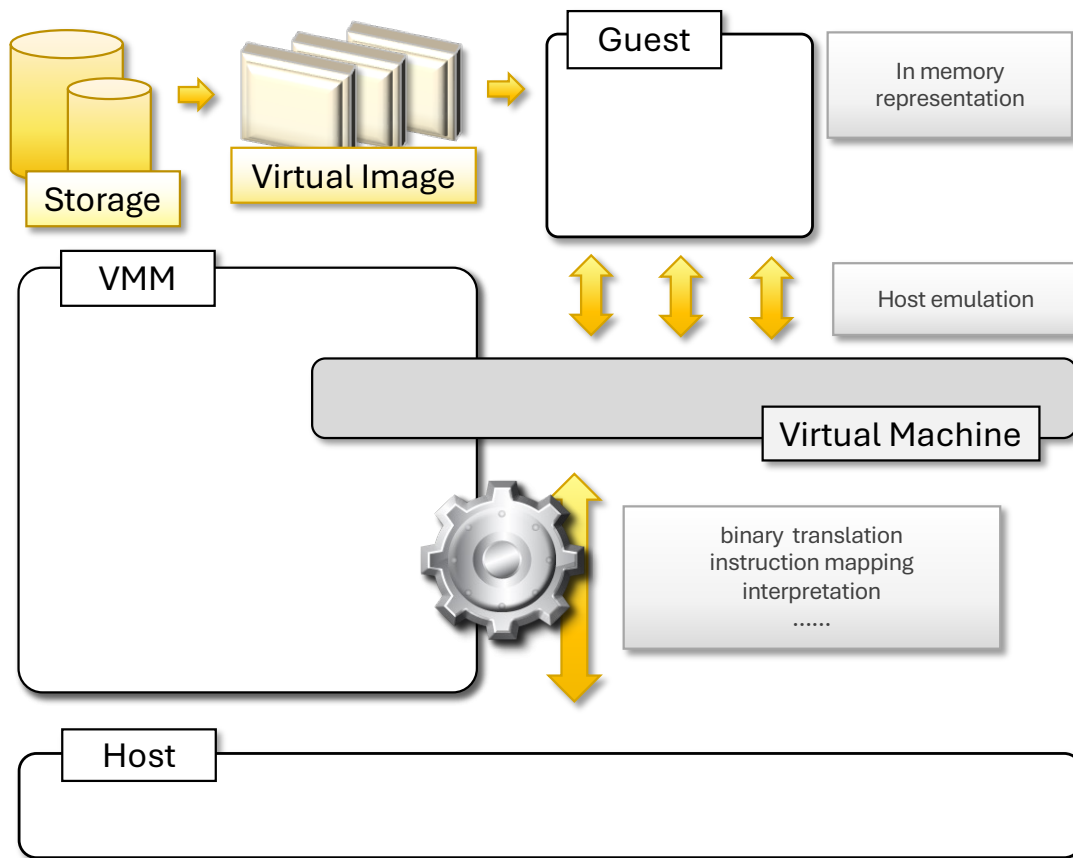
▪ **Instruction Set Architecture (ISA):** Defines instruction set for processor, registers, memory, interrupt management. Interface between hardware and software.

▪ **Application Binary Interface (ABI):** Separates OS from applications/libraries, covers low-level data types, call conventions, defines executable format. Defines system calls.

▪ **Application Programming Interface (API):** Interfaces applications to libraries and/or OS.

▪ **Security Classes/Privilege Modes:** Instruction set divided into privileged (sensitive operations, I/O, CPU state alteration) and nonprivileged (arithmetic, etc.). Hierarchy of privileges (Ring 0, 1, 2, 3) where Ring 0 is most privileged (kernel) and Ring 3 least (user). Most modern systems use supervisor (Ring 0) and user (Ring 3) modes. Hypervisor conceptually runs "above" supervisor mode (Ring -1 in newer hardware).

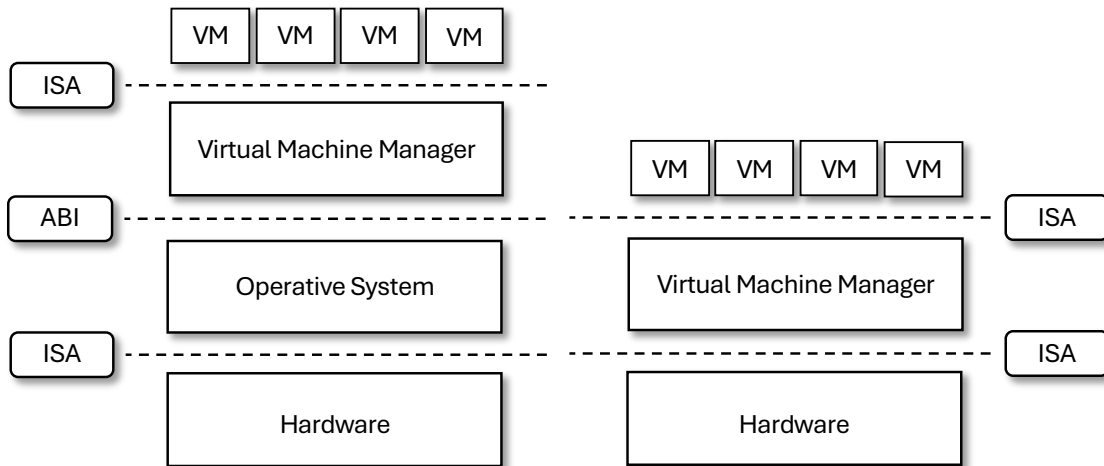
◦ **Hardware-Level Virtualization (System Virtualization):** Provides an abstract execution environment in terms of computer hardware. Guest is the OS, host is physical hardware, virtual machine is the emulation, and VMM is the hypervisor. Exposes ISA to virtual machines.



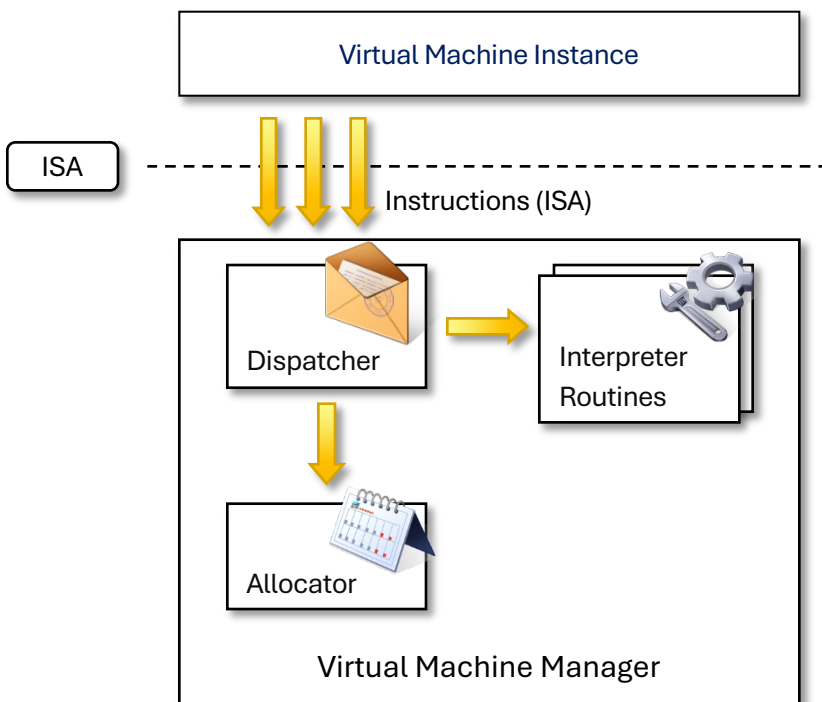
- **Hypervisors (VMM):** Recreates a hardware environment for guest OS.

- **Type I (Native/Bare-metal):** Runs directly on hardware, replacing the OS, interacts directly with ISA. Examples: VMware ESX/ESXi, Xen (in some configurations), Hyper-V.

- **Type II (Hosted):** Requires an operating system, runs as a program managed by the OS, emulates ISA for guests. Examples: VMware Workstation, VirtualBox.



▪ **Hypervisor Architecture:** Composed of Dispatcher (reroutes instructions), Allocator (decides resources for VM), and Interpreter (executes privileged instructions upon trap).



▪ **Popek and Goldberg Criteria (1974):** VMM must satisfy **Equivalence** (guest behaves same as on physical host), **Resource Control** (VMM has complete control of virtualized resources), and **Efficiency** (statistically dominant fraction of instructions execute without VMM intervention). **Theorem 3.1** states VMM construction is possible if sensitive instructions are a

subset of privileged ones. **Theorem 3.2** defines recursive virtualizability. **Theorem 3.3** describes hybrid VMM construction.

- **Hardware Virtualization Techniques:**

- **Hardware-Assisted Virtualization:** Hardware provides architectural support for VMM (e.g., Intel VT, AMD V). Reduces performance penalties of emulating x86 hardware. Solved x86 architecture not meeting Popek and Goldberg requirements.

- **Full Virtualization:** Runs an unmodified OS directly on a virtual machine, as if on raw hardware, providing complete emulation of underlying hardware. Advantages: complete isolation, enhanced security, ease of emulation, coexistence of systems. Challenges: performance, interception of privileged instructions. Achieved through hardware-assisted virtualization or **dynamic binary translation** (VMware).

- **Paravirtualization:** Not-transparent solution where VMM exposes a slightly modified software interface, requiring guests to be modified. Aims to execute performance-critical operations directly on host, preventing performance loss. Simpler VMM implementation. Common in open-source/academic environments (e.g., Xen modifies Linux kernels, uses hypercalls for sensitive operations). Can use ad hoc device drivers for unportable OS (e.g., Xen for Windows).

- **Partial Virtualization:** Provides partial emulation of hardware, not allowing complete isolation. Example: address space virtualization in time-sharing systems, where applications share hardware but have separate memory space.

- **Operating System-Level Virtualization:** Creates different, separated execution environments for applications within a **single operating system**. No VMM/hypervisor; OS kernel manages multiple isolated user space instances, sharing system resources and limiting interference. Instances have isolated file systems, separate IP addresses, configs, device access. Evolution of Unix chroot mechanism. Low overhead as applications use OS system calls directly. Lacks flexibility of hardware virtualization as all instances share same OS. Efficient for server consolidation of similar application servers. Examples: FreeBSD Jails, Solaris Zones, Parallels Virtuozzo Containers, OpenVZ.

- **Programming Language-Level Virtualization (Process Virtual Machines):** Uses a virtual machine to execute program **bytecode** (result of compilation). Achieves ease of deployment, managed execution, and portability across platforms. Bytecode is interpreted or Just-In-Time (JIT) compiled at runtime [224, 226n]. Examples: Java (JVM), .NET Framework (CLI). Stack-based (Java, CLI) or register-based (Parrot).

- **Advantages:** Uniform execution environment, simplifies development/deployment, control over program execution, security (sandboxing, pluggable security policies).

- **Disadvantage:** Performance overhead (getting smaller).

- **Application-Level Virtualization:** Runs applications in runtime environments that don't natively support all features required. Emulates partial file systems, libraries, OS components. Can execute binaries compiled for different hardware via **interpretation** (poor performance, minimal startup) or **binary translation** (initial overhead, better performance after caching).

- **Advantages:** Good for missing libraries, lighter VMM than hardware virtualization, allows incompatible apps to run together.

- **Examples:** Wine (Unix for Windows apps), CrossOver (Mac OS X for Windows apps), VMware ThinApp (packages installed apps into isolated executable images).

- **Other Types of Virtualization:**

- **Storage Virtualization:** Decouples physical organization of hardware from logical representation, allowing data to be identified by a logical path. Aggregates diverse storage facilities under a single logical file system (e.g., Storage Area Networks - SANs).

- **Network Virtualization:** Combines hardware/software to create and manage virtual networks.

- **External:** Aggregates physical networks into single logical network (e.g., Virtual LAN - VLAN).

- **Internal:** Provides network-like functionality to OS partition, often with hardware/OS-level virtualization (guests get virtual network interface). Options include shared host interface (NAT), emulated network device, private network only with guest.

- **Desktop Virtualization:** Abstracts desktop environment to provide access via client/server approach. Desktop environment stored remotely (server/datacenter), accessed via network. Provides high availability, persistence, accessibility, ease of management. Hardware virtualization infrastructure is fundamental for hosting multiple desktop environments on one server. Examples: Windows Remote Services, Sun VDI, Citrix XenDesktop.

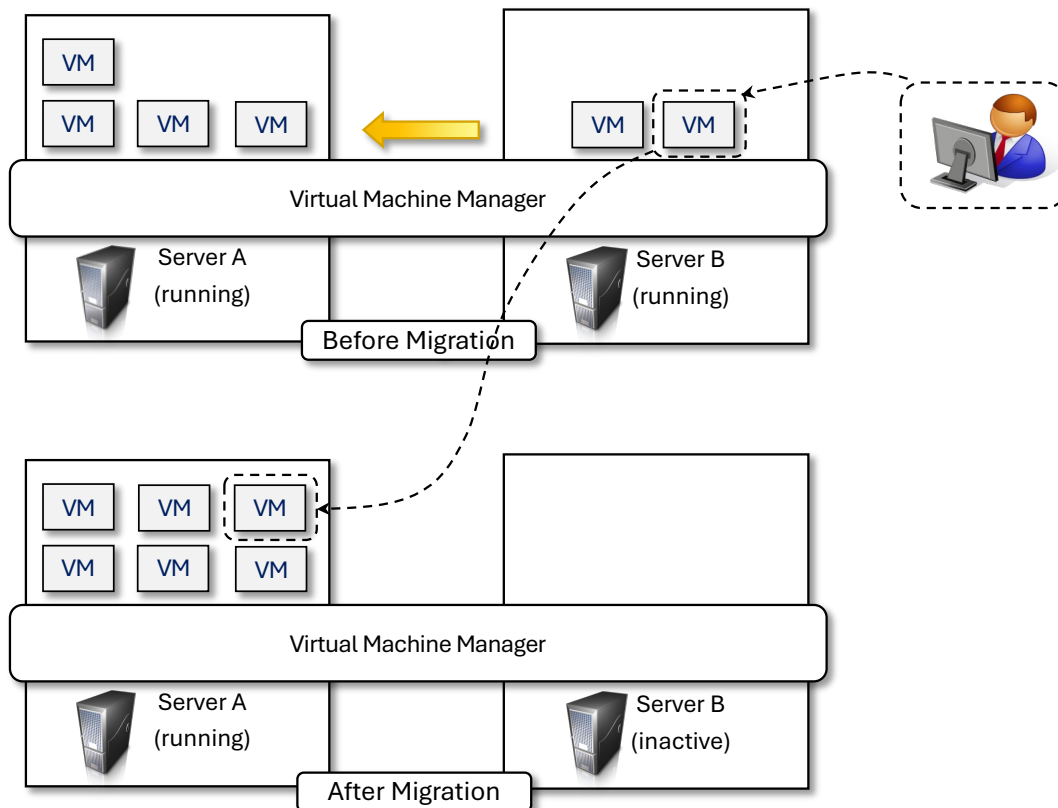
- **Application Server Virtualization:** Abstracts a collection of application servers into a single virtual application server using load-balancing and providing high-availability. Focuses on QoS rather than emulation.

Virtualization and Cloud Computing

- **Enabler for Cloud Services:** Virtualization provides the necessary **customization, security, isolation, and manageability** for delivering IT services on demand. Primarily used for configurable computing environments and storage.

- **Key Techniques in Cloud:**

- **Hardware Virtualization:** Enabling factor for **Infrastructure-as-a-Service (IaaS)**.
- **Programming Language Virtualization:** Leveraged in **Platform-as-a-Service (PaaS)** offerings.
- **Business Opportunity:** Capability to offer customizable and sandboxed environments is attractive for companies with large computing infrastructure. Also simplifies leasing services and accountability.
- **Efficient System Design:** Virtualization allows designing more efficient computing systems through **consolidation**, transparently to users. Since isolated/controllable environments can share resources without interference, it's attractive when resources are underutilized.
 - **Server Consolidation:** Aggregates virtual machines over fewer, fully utilized physical resources, reducing active resources and saving energy.
 - **Virtual Machine Migration:** Movement of virtual machine instances for consolidation. **Live migration** (moving while running) is more complex but more efficient, causing no service disruption [252, 253n].
- **Revamped Concepts:**
 - **Storage Virtualization:** Vendors with huge storage facilities can harness them into partitionable, dynamic virtual storage services.
 - **Desktop Virtualization:** Cloud computing revamps this concept, enabling a complete virtual computer hosted by a provider and accessed by a thin client over the Internet.



Pros and Cons of Virtualization

Pros: efficient resource use, scalability, portability, easier testing and recovery.

Cons: performance overhead, hypervisor vulnerabilities, complexity in management.

Advantages

- **Managed Execution and Isolation:** Allows building **secure and controllable computing environments**. Virtual environments configured as sandboxes prevent harmful operations. Simplified resource allocation and partitioning, enabling fine-tuning for server consolidation and QoS.
- **Portability:** Virtual machine instances are typically files, easily transported and self-contained (few dependencies beyond VMM). Java programs "compiled once and run everywhere". Enables migration techniques in server consolidation.
- **Reduced Maintenance Costs:** Fewer physical hosts mean lower maintenance burden, as guest programs have limited ability to damage underlying hardware.

- **Efficient Resource Use:** Multiple systems can securely coexist and share host resources without interference. Prerequisite for server consolidation, dynamically adjusting active physical resources to current load, leading to energy savings and environmental benefits.

Disadvantages

- **Performance Degradation:**

- **Increased Latencies:** Virtualization interposes an abstraction layer between guest and host.
- **Overhead:** Caused by maintaining virtual processor status, supporting privileged instructions (trap and simulate), paging within VM, console functions.
- **Scheduling:** If VMM runs on host OS (Type II), it shares resources with other applications, causing performance degradation.
- **Binary Translation/Interpretation:** In programming-level VMs, this can slow down execution.
- **Mitigation:** Technology advancements (e.g., paravirtualization, hardware-assisted virtualization, JIT compilation) are continuously reducing this impact.

- **Inefficiency and Degraded User Experience:**

- **Inaccessible Host Features:** Abstraction layer may not expose all specific host features (e.g., device drivers, graphics card capabilities).
- **Limited Features:** Early Java had limited graphics support compared to native applications.

- **Security Holes and New Threats:**

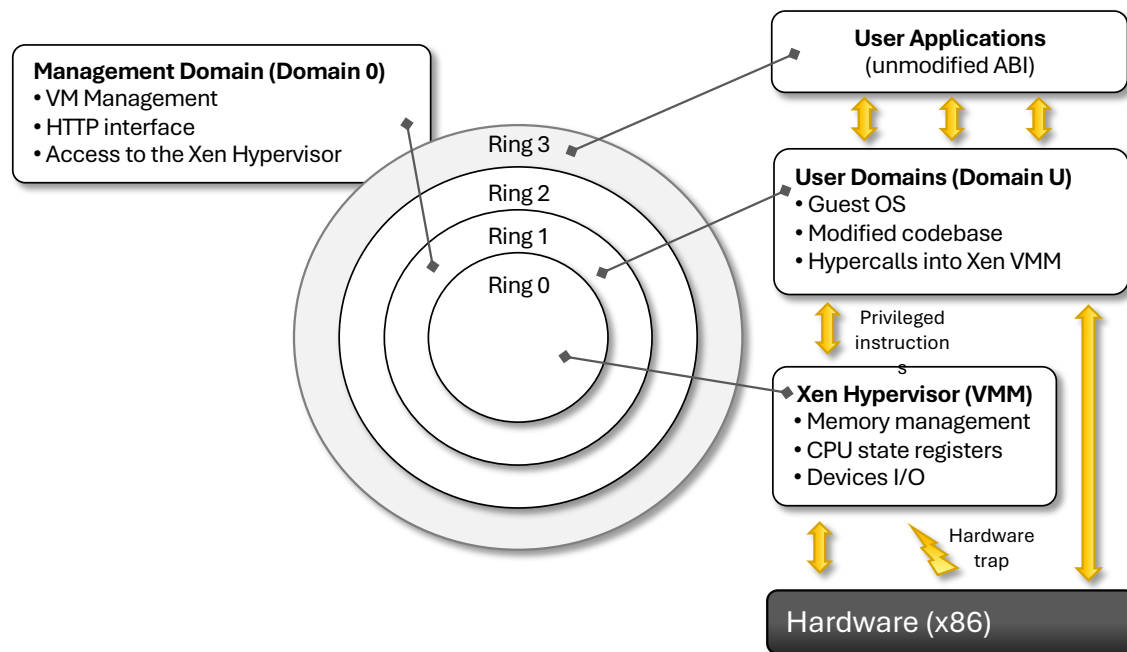
- **Phishing/Malware:** Ability to transparently emulate a host can lead to malicious programs designed to extract sensitive information from guests [269, 273n].
- **Malicious VMMs:** Programs like BluePill (AMD) and SubVirt (Microsoft prototype) can preload before the OS, acting as VMMs to control/manipulate the OS and extract data. Facilitated by older hardware/CPUs not designed with virtualization security in mind.
- **Programming-Level VMs:** Modified runtime environments can access sensitive information or monitor memory locations; requires malware running in administrative context or exploiting OS security holes.

Technology Examples – VMware and Microsoft Hyper-V

- **VMware:** Pioneer of full virtualization, allows multiple OS isolation.
- **Microsoft Hyper-V:** Windows-based hypervisor for enterprise use.
- **Xen:** Open-source paravirtualization.

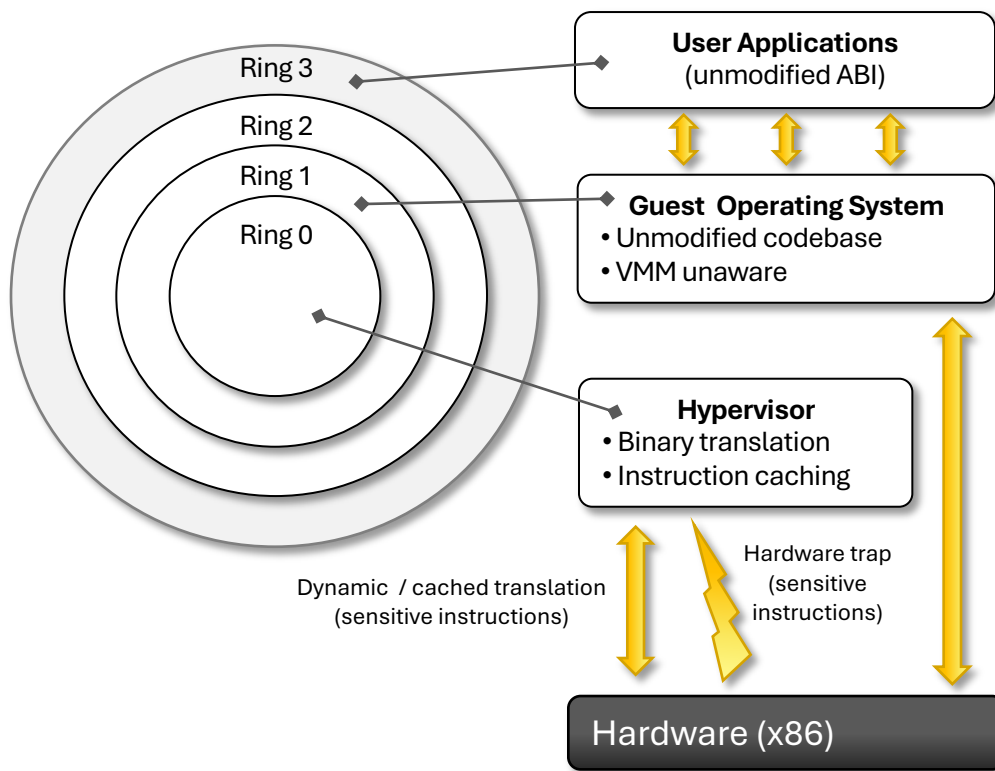
Xen: Paravirtualization

- **Overview:** Open-source virtualization platform based on paravirtualization. Developed at University of Cambridge, backed by a large community, commercial solution (Citrix XenSource). Used for desktop, server, and cloud computing (Xen Cloud Platform - XCP). Now supports full virtualization with hardware-assisted virtualization.
- **Mechanism:** Achieves high-performance by modifying portions of guest OS to eliminate performance loss during special instruction execution. Not a transparent virtualization solution.
- **Architecture:**
 - **Xen Hypervisor:** Runs in the highest privileged mode (Ring 0, or Ring -1 with hardware-assisted virtualization). Controls guest OS access to hardware.
 - **Domains:** Guest OS executed within domains (virtual machine instances).
 - **Domain 0 (Management Domain):** Special privileged domain running control software, loaded first after hypervisor boot. Hosts an HTTP server for virtual machine creation, configuration, termination requests. Serves as embryonic distributed VMM for IaaS cloud systems.
 - **Domain U (User Domain):** Other domains running guest OS (typically in Ring 1 or 0 with hardware assistance). User applications run in Ring 3, maintaining ABI unchanged.
 - **Hypercalls:** Guest OS needs to be modified to use hypercalls (specific calls exposed by Xen's virtual machine interface) to handle sensitive system calls that would otherwise trap. Xen catches these, manages them, and returns control to the guest.
- **Limitations:** Requires modified OS codebase; thus, not all OS can be guests in Xen-based environments without hardware-assisted virtualization. Legacy hardware/OS cannot be modified or run safely in Ring 1. Open-source OS (Linux) are easily modified; Windows generally not supported without hardware-assisted virtualization.



VMware: Full Virtualization

• **Overview:** Technology based on **full virtualization**, replicating underlying hardware for unmodified guest OS. Implements full virtualization via **direct execution** (for nonsensitive instructions) and **binary translation** (for sensitive instructions). Offers desktop (Type II hypervisors) and server (Type I hypervisors) solutions. Provides tools for integration and infrastructure management.



• Full Virtualization and Binary Translation:

◦ **x86 Virtualization:** x86 architecture did not originally satisfy Popek and Goldberg's first theorem (sensitive instructions not subset of privileged). Older VMware products (before hardware-assisted virtualization in 2006) used **dynamic binary translation** to run unmodified x86 guest OS.

◦ **Mechanism:** When sensitive instructions cause a trap, they are translated into an equivalent set that avoids exceptions, and the translated instructions are cached for performance.

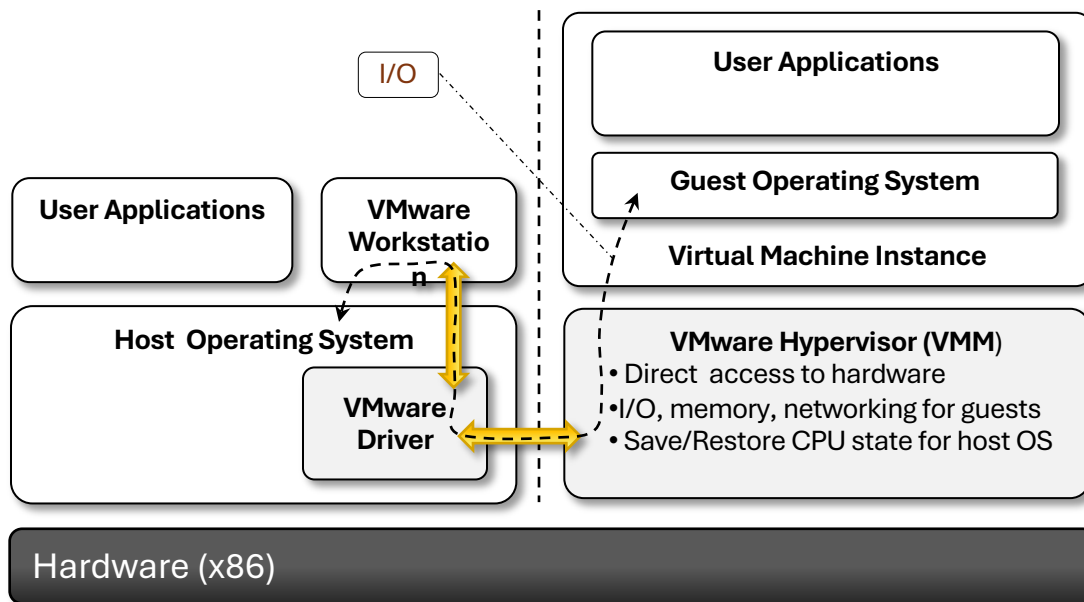
◦ **Advantages:** Guests run unmodified, crucial for OS without source code (e.g., Windows). More portable solution for full virtualization.

◦ **Disadvantage:** Runtime translation introduces overhead, though reduced by direct execution of other instructions.

• **Memory and I/O Virtualization:** Achieves full virtualization of memory and I/O devices. Memory virtualization is challenging due to MMU emulation; TLB (translation look-aside buffer) direct mapping reduces impact. Provides full virtualization of network controllers, peripherals (keyboard, mouse, disks, USB).

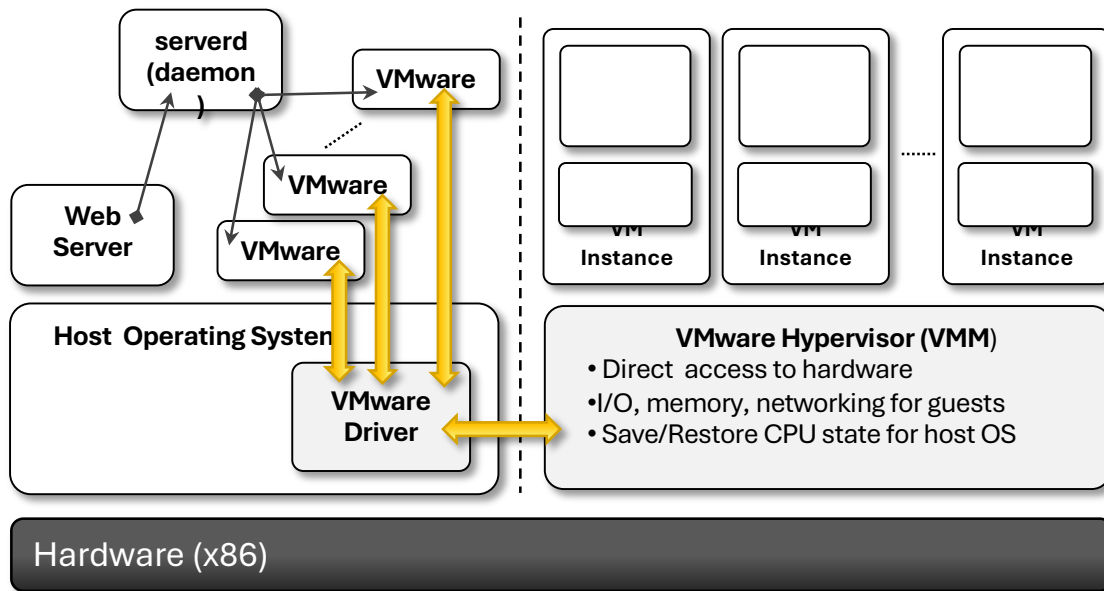
• Virtualization Solutions:

◦ **End-User (Desktop) Virtualization:** VMware Workstation (Windows) and VMware Fusion (Mac OS X) install as applications on host OS, creating and managing virtual machines. Provide full hardware virtualization via a specific driver in host OS that runs VMM in privileged mode and handles I/O requests. Virtual machine images saved as files, with features like snapshots, rollback. Other products: VMware Player (reduced Workstation), VMware ACE (policy-wrapped VMs), VMware ThinApp (application virtualization for isolated apps).

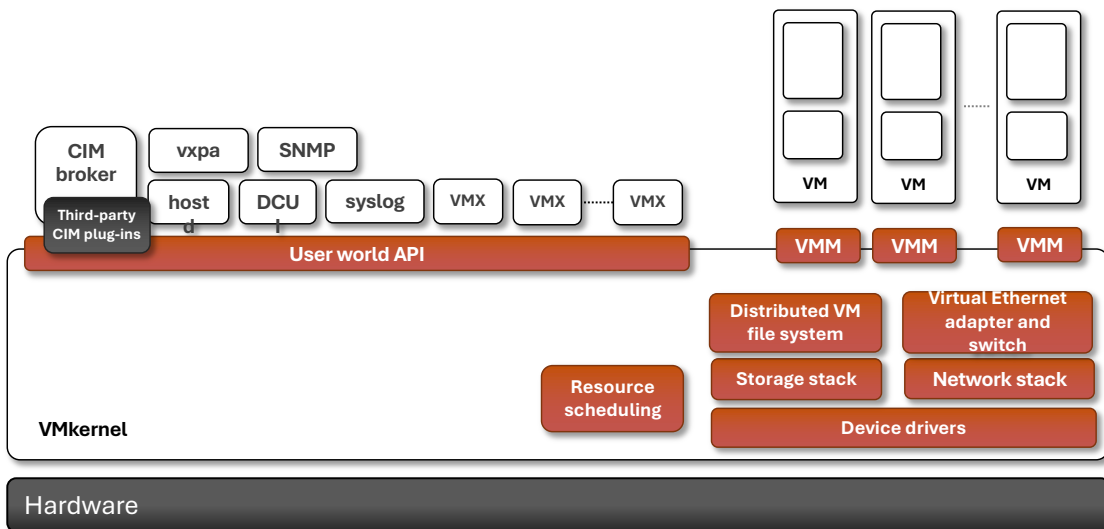


◦ **Server Virtualization:**

▪ **VMware GSX Server:** Replicates desktop approach for servers, adding remote management. Daemon serverd controls VMware app processes, connected to VM instances via VMware driver.



• **VMware ESX Server & ESXi Server:** Type I hypervisors installed on bare metal. ESX embeds modified Linux (service console for hypervisor access), ESXi has a very thin OS layer with remote management interfaces.



• **ESXi Architecture:** Base is **VMkernel** (thin POSIX-compliant OS for process/thread management, file system, I/O stacks, resource scheduling). Accessible via User World API. Remote management via **CIM Broker** (gateway to VMkernel using CIM protocol). Local management via **Direct Client User Interface (DCUI)**.

• **Infrastructure Virtualization and Cloud Solutions:**

- **VMware vSphere:** Management of virtual infrastructure; a pool of virtualized servers managed by vSphere. Provides basic services (virtual file system, storage, network) and application services (VM migration, storage migration, data recovery, security zones).

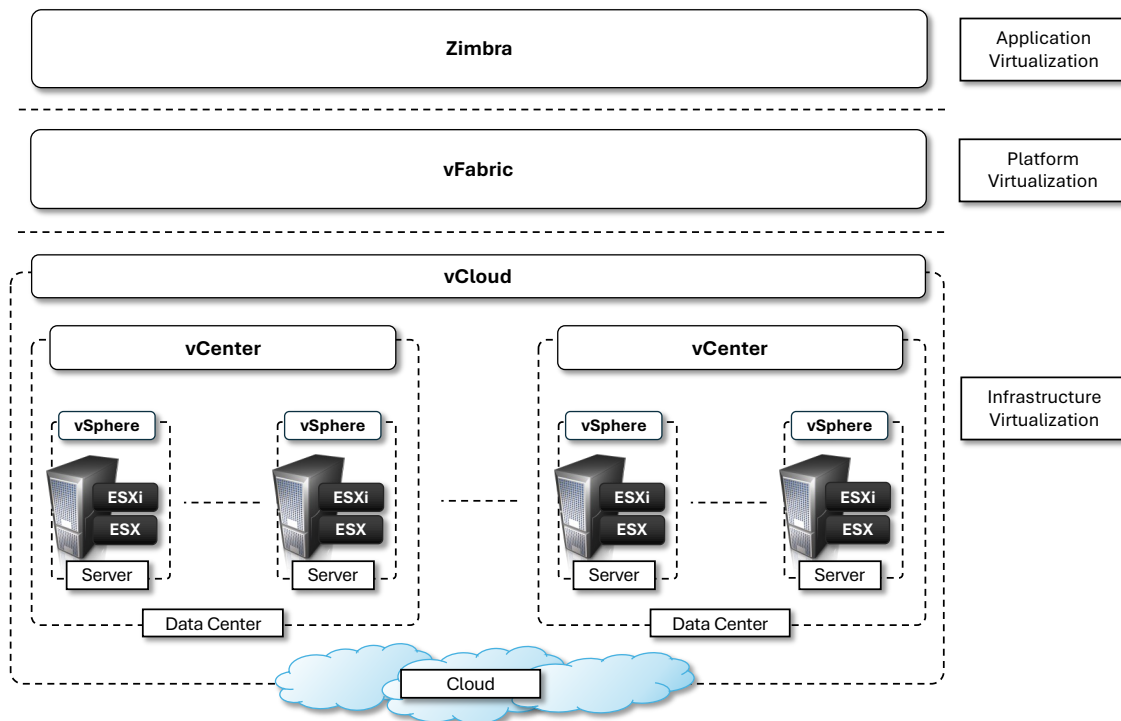
- **VMware vCenter:** Centralized administration and management of vSphere installations in data centers.

- **VMware vCloud:** Turns virtualized data centers into an IaaS cloud, allowing providers to offer on-demand virtual computing environments on a pay-per-use basis. Web portal for self-provisioning VMs and setting up virtual networks.

- **VMware vFabric:** Platform for application development in the cloud, components for scalable Web applications on virtualized infrastructure (monitoring, data management, Java Web app execution/provisioning).

- **Zimbra:** SaaS solution for office automation, messaging, collaboration, hosted in the cloud.

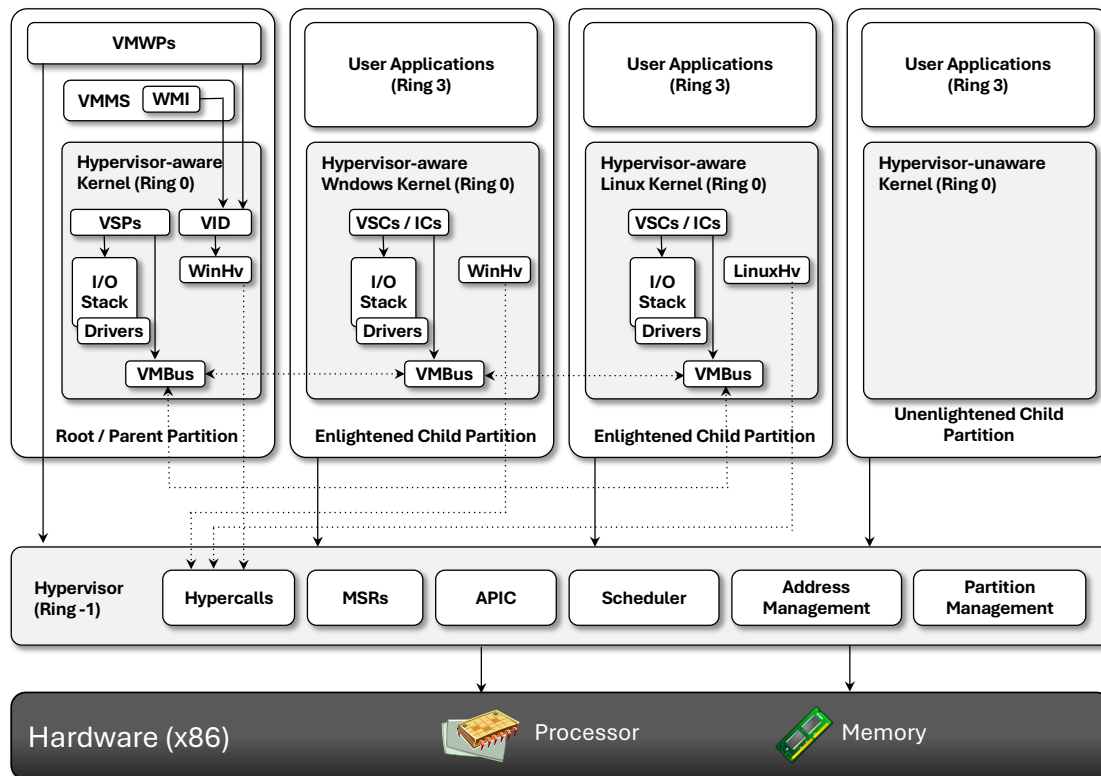
• **Observations:** VMware started with full x86 virtualization but integrated paravirtualization features (e.g., VMware Tools, VMI - vendor-independent Virtual Machine Interface).



Microsoft Hyper-V

- **Overview:** Infrastructure virtualization solution for server virtualization developed by Microsoft. Hypervisor-based approach leveraging various techniques to support guest OS. Shipped as a component (role) within Windows Server 2008 R2.
- **Architecture:** Supports multiple concurrent guest OS via **partitions**(isolated environments). Hyper-V takes control of hardware, and the host OS becomes a special privileged virtual machine instance called the **parent partition** (or root partition).
 - **Parent Partition:** Direct access to hardware, runs virtualization stack, hosts drivers for guest OS, creates child partitions via hypervisor. Always hosts a Windows Server 2008 R2 instance. Manages child partition creation/execution/destruction via **Virtualization Infrastructure Driver (VID)**. Instantiates a **Virtual Machine Worker Process (VMWP)** for each child partition. Accessible remotely via WMI provider.
 - **Child Partitions:** Host guest OS, no direct hardware access. Interaction controlled by parent partition or hypervisor. Two types: **Enlightened** (Hypervisor-aware, benefit from Enlightened I/O) and **Unenlightened** (Hypervisor-unaware, rely on less efficient device driver emulation).
- **Hypervisor:** Component directly managing underlying hardware (processors, memory). Runs in **Ring -1**, requiring hardware support.
 - **Hypercalls Interface:** Entry point for partitions to execute sensitive instructions (paravirtualization approach). Used by drivers and parent partition.
 - **Memory Service Routines (MSRs):** Control memory access from partitions, leverage hardware-assisted virtualization (I/O MMU or IOMMU) for fast device access.
 - **Advanced Programmable Interrupt Controller (APIC):** Manages hardware signals/interrupts. Virtual processors have **Synthetic Interrupt Controller (SynIC)**.
 - **Scheduler:** Schedules virtual processors on physical processors, controlled by parent partition policies.
 - **Address Manager:** Manages virtual network addresses for guests.
 - **Partition Manager:** Creates, finalizes, destroys, enumerates, configures partitions.
- **Enlightened I/O and Synthetic Devices:** Optimized I/O method allowing hypervisor-aware guests to use an interpartition communication channel (**VMBus**) instead of hardware emulation stack.
 - **Components:** VMBus (channel/protocol), Virtual Service Providers (VSPs - kernel-level drivers in parent partition providing hardware access), Virtual Service Clients (VSCs - virtual device drivers in child partitions).

◦ **Benefit:** Enhanced performance for I/O (storage, networking, graphics, input). Also improves child-to-child I/O via virtual networks.



• Cloud Computing and Infrastructure Management:

◦ **Windows Server Core:** Reduced version of Windows Server 2008 with fewer features (no GUI, .NET Framework) for reduced maintenance, attack surface, management, and disk space. Managed remotely via PowerShell and WMI.

◦ **System Center Virtual Machine Manager (SCVMM) 2008:** Component of Microsoft System Center suite for virtual infrastructure management. Complements Hyper-V with management capabilities: portal for virtual instance creation/management, V2V/P2V conversions, delegated administration, library functions, PowerShell integration, intelligent VM placement, host capacity management. Works with other platforms (VMware vSphere) but optimized for Hyper-V.

• **Observations:** Hyper-V is a **hybrid solution**, leveraging both paravirtualization (hypercalls, VMBus) and full hardware virtualization. Parent/child partitions are conceptually similar to Xen's Domain 0/U. Hyper-V installed as a role in existing OS (like hosted hypervisor) while Xen installed on bare hardware. Requires specific hardware (x64, hardware-assisted virtualization, DEP) and Windows Server 2008 or newer.

Before the Move into the Cloud

Know Your Software Licenses

Organizations must audit software licenses before migration. Issues include BYOL (Bring Your Own License), hardware/CPU-based restrictions, and license mobility in cloud environments.

The Shift to a Cloud Cost Model

Shift from CapEx (buying servers upfront) to OpEx (pay-as-you-go). Benefits include reduced upfront costs, flexibility. Challenges: unpredictable costs, need for monitoring and optimization.

Service Levels for Cloud Applications

Cloud providers define SLAs (availability, latency, support response). Customers must assess if SLA commitments match their business needs.